```
Output:
c581756b61737a
5a656265646565
```

Remember the binary comparison rules: the two strings differ at the very first byte, and of course *0xc5* is greater than *0x5a* (by decimal *107*, in fact).

Technically `strcmp()` and `strnatcmp()` are multi-byte safe because they are not tools for comparing characters; they are designed only for the comparison of arrays of bytes. `strcasecmp()` and `strnatcasecmp()` will fail hard, however, once outside the ASCII range (A–Z, a–z), because they only contain logic for dealing with the basic Latin alphabet: All that's happening is bytes *0x41–0x5a* (A–Z) map to bytes *0x61–0x7a* (a–z). For example:

```
$A = "\u{41}"; // Unicode codepoint for A
$a = "\u{61}"; // a

$latinCapital_L_WithStroke = "\u{141}"; // Ł
$latinSmall_L_WithStroke = "\u{142}"; // ł

var_dump(strcasecmp($A, $a));
var_dump(strcasecmp(
    $latinCapital_L_WithStroke,
    $latinSmall_L_WithStroke
));
```

```
Output:
int(0)
int(-1)
```

Unicode codepoints *0x41* and *0x61* are ASCII characters; they're mapped to each other and so return `0` (no difference). The Polish character Ł and its lowercase form ł lie way outside the ASCII range, `strcasecmp()` has no knowledge of Unicode, and the result is `-1`.

There is no multi-byte safe form of `strcasecmp()` or `strnatcasecmp()`, but one way to get around this is to make use of `mb_strtolower()` before calling `strcasecmp()`.

```
$polishChars1 = "\u{141}\u{143}\u{146}";
$polishChars2 = "\u{142}\u{144}\u{145}";
```